

Universidad de Monterrey



Integración de aplicaciones computacionales

Ejercicio Guiado 5

**Microservicios en Flask con JWT usando Redis**

**Maestro:** Dr. Raúl Morales Salcedo

**Nombre:** Margarita Concepción Cuervo Citalán #581771

**Carrera:** ITC, 9no Semestre

**Día y hora del grupo de la clase:** lunes y jueves 10:00h

11 de octubre de 2025.

*Doy mi palabra de haber realizado esta actividad con integridad académica.*

# ÍNDICE

|                             |    |
|-----------------------------|----|
| Introducción                | 2  |
| Desarrollo                  | 2  |
| Consideraciones importantes | 43 |
| Conclusión                  | 44 |

## Introducción

Este ejercicio guiado tuvo como objetivo el diseño e implementación de una arquitectura de microservicios para un sistema de gestión de un catálogo de libros, con especial énfasis en la seguridad y control de sesiones. A partir de una base previa con Flask, JWT y MariaDB, se integró Redis como una capa de soporte para manejar la gestión de tokens de acceso y refresh, revocación de sesiones, y validación estricta mediante listas allowlist y denylist. Esta mejora permitió fortalecer el control de autenticación en todos los endpoints del sistema.

La solución se desplegó en una máquina virtual en Google Cloud Platform, y se utilizó un cliente web (HTML, CSS, JavaScript) para interactuar con el backend. Esta comunicación fue reforzada por JWT y Redis, asegurando que únicamente los usuarios autorizados pudieran interactuar con las rutas protegidas del API. Además, se ajustó el archivo `script.js` del cliente para actualizar correctamente las rutas y asegurar la comunicación con los nuevos endpoints definidos en el microservicio.

## Desarrollo

Se empezó este ejercicio por crear una copia del subdirectorio *micro01* dentro de *Microservices*, llamado *micro02*. Dentro de este subdirectorio se creó previamente un microservicio que ya se conectaba con la base de datos Libros y tenía un front que se conectaba con el back y los endpoints usando JWT como método de autenticación. El prompt usado fue el siguiente:

*Partiendo del ejercicio guiado de autenticación con Flask + MariaDB + JWT + Cliente Web, deberás migrar/ajustar la autenticación para que se gestione mediante Redis, y aplicarla a todos los endpoints del microservicio de Libros construido en sesiones previas.*

*Objetivo: Diseñar y desplegar un microservicio de Libros (Books API) donde todas las rutas estén protegidas por JWT y donde Redis sea la capa de soporte para:*

- *Gestión de sesiones y tokens (allowlist/denylist).*
- *Revocación de tokens.*
- *Refresh de Access Tokens.*
- *(Opcional) Rate limiting por usuario/endpoint.*

*Debes implementar los siguientes endpoints ya definidos en sesiones pasadas, ahora todos protegidos con JWT:*

- *GET /api/books → muestra todos los libros en formato XML.*
- *GET /api/books/ISBN → muestra un libro si se manda el ISBN.*
- *GET /api/books/format/ → muestra todos los libros su formato.*

- *GET /api/books/autor/ → muestra todos los libros de un autor.*
- *POST /api/books/create → inserta un nuevo libro.*
- *PUT /api/books/update → actualiza un libro.*
- *DELETE /api/books/delete → elimina un libro si se manda el ISBN.*

*Requisito: Todos estos endpoints deben rechazar peticiones sin JWT o con tokens inválidos/expirados (responder con 401/403).*

*Debes incluir también los endpoints de autenticación:*

- *POST /auth/register → registra usuario (hash de contraseña).*
- *POST /auth/login → devuelve access\_token y refresh\_token.*
- *POST /auth/refresh → emite un nuevo access token usando refresh válido.*
- *POST /auth/logout → revoca el token actual (denylist en Redis).*

Al ejecutarlo, la IA nos dio el contenido editado del archivo ejecutable **main.py**, el cual contiene el siguiente código:

```
Python
from flask import Flask, request, jsonify, Response, make_response
from flask_cors import CORS
from datetime import timedelta, datetime, timezone
from flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    jwt_required, get_jwt_identity, get_jwt
)
from flask_jwt_extended.utils import decode_token
from passlib.hash import pbkdf2_sha256
import MySQLdb
import xml.etree.ElementTree as ET
import redis
import os

# =====
# App & Config
# =====
app = Flask(__name__)

# JWT
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY',
'super-secret-key-change-me')
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)
app.config['JWT_TOKEN_LOCATION'] = ['headers']
app.config['JWT_HEADER_NAME'] = 'Authorization'
```



```

app.config['JWT_HEADER_TYPE'] = 'Bearer'
jwt = JWTManager(app)

# CORS (abierto para pruebas)
CORS(
    app,
    resources={r"/*": {"origins": "*"}},
    methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["Content-Type", "Authorization"],
    expose_headers=["Content-Type", "Authorization"],
    supports_credentials=False,
    send_wildcard=True,
    max_age=86400,
)

# MariaDB
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_USER = os.getenv("DB_USER", "libro_user")
DB_PASS = os.getenv("DB_PASS", "666")
DB_NAME = os.getenv("DB_NAME", "Libros")
DB_CHARSET = "utf8"

def get_db():
    return MySQLdb.connect(
        host=DB_HOST, user=DB_USER, passwd=DB_PASS,
        db=DB_NAME, charset=DB_CHARSET
    )

# Redis
REDIS_HOST = os.getenv("REDIS_HOST", "127.0.0.1")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))
REDIS_DB = int(os.getenv("REDIS_DB", "0"))
r = redis.Redis(host=REDIS_HOST, port=REDIS_PORT, db=REDIS_DB, decode_responses=True)

# =====
# Helpers: JWT + Redis
# =====
def _now_ts():
    return int(datetime.now(timezone.utc).timestamp())

def _ttl_from_exp(exp_unix: int) -> int:
    ttl = exp_unix - _now_ts()
    return ttl if ttl > 0 else 1

def _allow_key(token_type: str, jti: str) -> str:
    return f"allow:{token_type}:{jti}"

def _block_key(jti: str) -> str:
    return f"block:{jti}"

def store_in_allowlist(encoded_token: str, token_type: str, username: str):
    decoded = decode_token(encoded_token)

```

```

    jti = decoded['jti']
    exp = decoded['exp']
    ttl = _ttl_from_exp(exp)
    key = _allow_key(token_type, jti)
    r.hset(key, mapping={"username": username, "type": token_type, "exp": str(exp)})
    r.expire(key, ttl)
    return jti

def is_in_allowlist(jti: str) -> bool:
    # válido si existe en allowlist (access o refresh)
    return r.exists(_allow_key("access", jti)) == 1 or r.exists(_allow_key("refresh",
jti)) == 1

def is_revoked(jti: str) -> bool:
    return r.exists(_block_key(jti)) == 1

def revoke_jti(jti: str, exp: int):
    ttl = _ttl_from_exp(exp)
    r.setex(_block_key(jti), ttl, "1")
    # opcional: limpiar allowlist
    r.delete(_allow_key("access", jti))
    r.delete(_allow_key("refresh", jti))

@jwt.token_in_blocklist_loader
def token_check_revoked(jwt_header, jwt_payload):
    # Política estricta: si NO está en allowlist -> inválido
    jti = jwt_payload["jti"]
    return is_revoked(jti) or (not is_in_allowlist(jti))

# Preflight (CORS)
@app.before_request
def handle_preflight():
    if request.method == "OPTIONS":
        resp = make_response()
        resp.headers["Access-Control-Allow-Origin"] = request.headers.get("Origin",
"*")
        resp.headers["Vary"] = "Origin"
        resp.headers["Access-Control-Allow-Headers"] = "Content-Type, Authorization"
        resp.headers["Access-Control-Allow-Methods"] = "GET,POST,PUT,DELETE,OPTIONS"
        return resp, 200

@app.after_request
def add_cors_headers(resp):
    resp.headers["Access-Control-Allow-Origin"] = request.headers.get("Origin", "*")
    resp.headers["Vary"] = "Origin"
    resp.headers["Access-Control-Allow-Headers"] = "Content-Type, Authorization"
    resp.headers["Access-Control-Allow-Methods"] = "GET,POST,PUT,DELETE,OPTIONS"
    return resp

# =====
# XML helpers
# =====

```

```

def books_to_xml(rows):
    catalog = ET.Element('catalog')
    for b in rows:
        book_elem = ET.Element('book', isbn=b.get('isbn', ''))
        title = ET.SubElement(book_elem, 'title'); title.text = b.get('titulo',
'Desconocido')
        # soporta múltiples autores (si hacen join con GROUP_CONCAT)
        authors_field = b.get('authors') or b.get('autor') or ''
        authors = authors_field.split(',') if authors_field else []
        if not authors:
            ET.SubElement(book_elem, 'author').text = 'Desconocido'
        else:
            for a in authors:
                ET.SubElement(book_elem, 'author').text = a.strip()

        year = ET.SubElement(book_elem, 'year'); year.text =
str(b.get('anio_publicacion', '0'))
        genre = ET.SubElement(book_elem, 'genre'); genre.text = b.get('genre_name',
b.get('genero', 'Desconocido'))
        price = ET.SubElement(book_elem, 'price'); price.text = str(b.get('price',
'0.0'))
        stock = ET.SubElement(book_elem, 'stock'); stock.text = str(b.get('stock',
'0'))
        fmt = ET.SubElement(book_elem, 'format'); fmt.text = b.get('formato',
'Desconocido')
        catalog.append(book_elem)
    return ET.tostring(catalog, encoding='utf-8', xml_declaration=True)

# =====
# AUTH
# =====

# POST /auth/register
@app.route('/auth/register', methods=['POST'])
def register():
    data = request.get_json(silent=True) or {}
    username = data.get('username'); password = data.get('password')
    if not username or not password:
        return jsonify({"msg": "username y password son requeridos"}), 400
    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("SELECT 1 FROM Users WHERE username=%s", (username,))
        if cur.fetchone():
            return jsonify({"msg": "Usuario ya existe"}), 409
        cur.execute("INSERT INTO Users (username, password_hash) VALUES (%s,%s)",
            (username, pbkdf2_sha256.hash(password)))
        conn.commit()
        return jsonify({"msg": "Usuario creado"}), 201
    except Exception as e:
        return jsonify({"msg": "DB error", "error": str(e)}), 500
    finally:
        try: conn.close()

```

```

        except: pass

# POST /auth/login
@app.route('/auth/login', methods=['POST'])
def login():
    data = request.get_json(silent=True) or {}
    username = data.get('username'); password = data.get('password')
    if not username or not password:
        return jsonify({"msg": "username y password son requeridos"}), 400
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("SELECT password_hash FROM Users WHERE username=%s", (username,))
        row = cur.fetchone()
        if not row or not pbkdf2_sha256.verify(password, row['password_hash']):
            return jsonify({"msg": "Credenciales inválidas"}), 401

        access = create_access_token(identity=username)
        refresh = create_refresh_token(identity=username)
        jti_a = store_in_allowlist(access, "access", username)
        jti_r = store_in_allowlist(refresh, "refresh", username)
        return jsonify(access_token=access, refresh_token=refresh,
                        jti_access=jti_a, jti_refresh=jti_r), 200
    except Exception as e:
        return jsonify({"msg": "DB/Auth error", "error": str(e)}), 500
    finally:
        try: conn.close()
        except: pass

# POST /auth/refresh
@app.route('/auth/refresh', methods=['POST'])
@jwt_required(refresh=True)
def refresh_token():
    user = get_jwt_identity()
    new_access = create_access_token(identity=user)
    jti_a = store_in_allowlist(new_access, "access", user)
    return jsonify(access_token=new_access, jti_access=jti_a), 200

# POST /auth/logout (revoca el token access actual)
@app.route('/auth/logout', methods=['POST'])
@jwt_required()
def logout():
    claims = get_jwt()
    revoke_jti(claims['jti'], claims['exp'])
    return jsonify({"msg": "logout ok", "revoked_jti": claims['jti']}), 200

# =====
# BOOKS (TODOS protegidos)
# =====

# GET /api/books → XML con todos
@app.route('/api/books', methods=['GET'])
@jwt_required()

```

```

def books_all():
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            GROUP BY l.isbn
        """)
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0' encoding='UTF-8'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# GET /api/books/ISBN → ?isbn=...
@app.route('/api/books/ISBN', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_isbn():
    isbn = request.args.get('isbn', '').strip()
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.isbn = %s
            GROUP BY l.isbn
        """, (isbn,))
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

```

```

# GET /api/books/format/ → ?format=...
@app.route('/api/books/format/', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_format():
    fmt = request.args.get('format', '').strip()
    if not fmt:
        return Response("<?xml version='1.0'?><error>format requerido</error>",
                        mimetype='application/xml', status=400)

    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.formato = %s
            GROUP BY l.isbn
        """, (fmt,))
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# GET /api/books/autor/ → ?name=...
@app.route('/api/books/autor/', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_autor():
    name = request.args.get('name', '').strip()
    if not name:
        return Response("<?xml version='1.0'?><error>name requerido</error>",
                        mimetype='application/xml', status=400)

    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE a.nombre LIKE %s
            GROUP BY l.isbn
        """, (f"%{name}%",))
        rows = cur.fetchall()
        xml = books_to_xml(rows)

```

```

        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# POST /api/books/create
@app.route('/api/books/create', methods=['POST'], strict_slashes=False)
@jwt_required()
def book_create():
    data = request.get_json(silent=True) or {}
    req = ['isbn', 'titulo', 'id_autor', 'id_categoria', 'id_editorial',
           'anio_publicacion', 'price', 'stock', 'formato']
    if any(k not in data for k in req):
        return Response("<?xml version='1.0'?><error>faltan campos</error>",
                        mimetype='application/xml', status=400)

    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("""
            INSERT INTO Libro (isbn, titulo, id_autor, id_categoria, id_editorial,
                               anio_publicacion, price, stock, formato)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
            """, (data['isbn'], data['titulo'], data['id_autor'], data['id_categoria'],
                  data['id_editorial'], data['anio_publicacion'], data['price'],
                  data['stock'], data['formato']))
        conn.commit()
        return Response("<?xml version='1.0'?><ok>inserted</ok>",
                        mimetype='application/xml', status=201)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# PUT /api/books/update
@app.route('/api/books/update', methods=['PUT'], strict_slashes=False)
@jwt_required()
def book_update():
    data = request.get_json(silent=True) or {}
    isbn = data.get('isbn')
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)

    fields = {k:v for k,v in data.items() if k in [
        'titulo', 'id_autor', 'id_categoria', 'id_editorial',
        'anio_publicacion', 'price', 'stock', 'formato'
    ] and v not in (None, '')}
    if not fields:
        return Response("<?xml version='1.0'?><error>nada que actualizar</error>",

```

```

        mimetype='application/xml', status=400)

try:
    conn = get_db(); cur = conn.cursor()
    sets = ", ".join([f"{k}=%s" for k in fields.keys()])
    params = list(fields.values()) + [isbn]
    cur.execute(f"UPDATE Libro SET {sets} WHERE isbn=%s", params)
    conn.commit()
    return Response("<?xml version='1.0'?><ok>updated</ok>",
                    mimetype='application/xml', status=200)
except Exception as e:
    err = f"<?xml version='1.0'?><error>{str(e)}</error>"
    return Response(err, mimetype='application/xml', status=500)
finally:
    try: conn.close()
    except: pass

# DELETE /api/books/delete → ?isbn=...
@app.route('/api/books/delete', methods=['DELETE'], strict_slashes=False)
@jwt_required()
def book_delete():
    isbn = request.args.get('isbn', '').strip()
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)

    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("DELETE FROM Libro WHERE isbn=%s", (isbn,))
        conn.commit()
        return Response("<?xml version='1.0'?><ok>deleted</ok>",
                        mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# Salud
@app.route('/ping', methods=['GET'])
def ping():
    return jsonify(msg="pong"), 200

if __name__ == '__main__':
    # Escucha en todas las interfaces
    app.run(host="0.0.0.0", port=5000, debug=True)

```

Se copió dicho código y con el comando `sudo vi main.py` se creó e insertó el código de arriba. Para poder visualizar el contenido de este en el navegador, se tuvo que crear una regla de firewall que permitiera acceder a cualquier IP de la red.

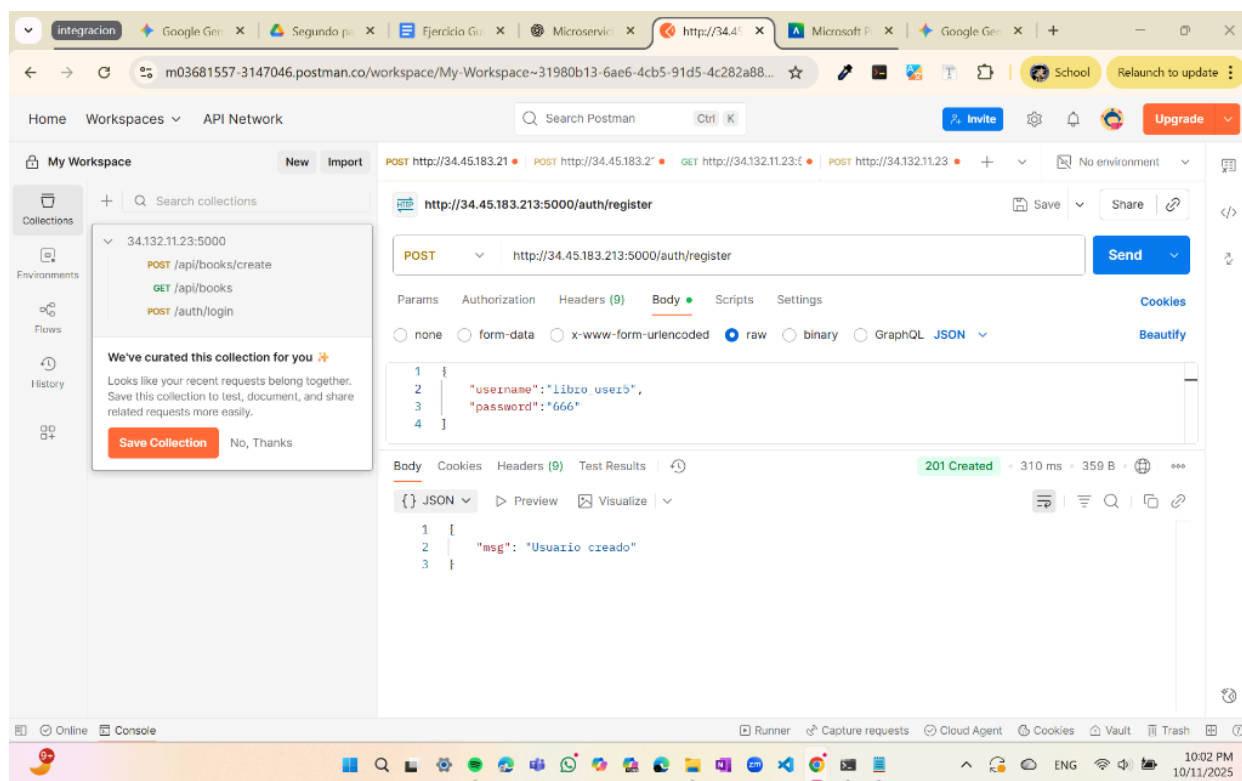


```
C:\Users\mccco>gcloud compute firewall-rules create api-flask-2 --network=default --direction=INGRESS --priority=1000 --
action=ALLOW --rules=tcp:5000-5005 --source-ranges=0.0.0.0/0
Creating firewall...Created [https://www.googleapis.com/compute/v1/projects/integracion-468300/global/firewalls/api-fla
sk-2].
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW         DENY  DISABLED
api-flask-2   default  INGRESS    1000      tcp:5000-5005  False
```

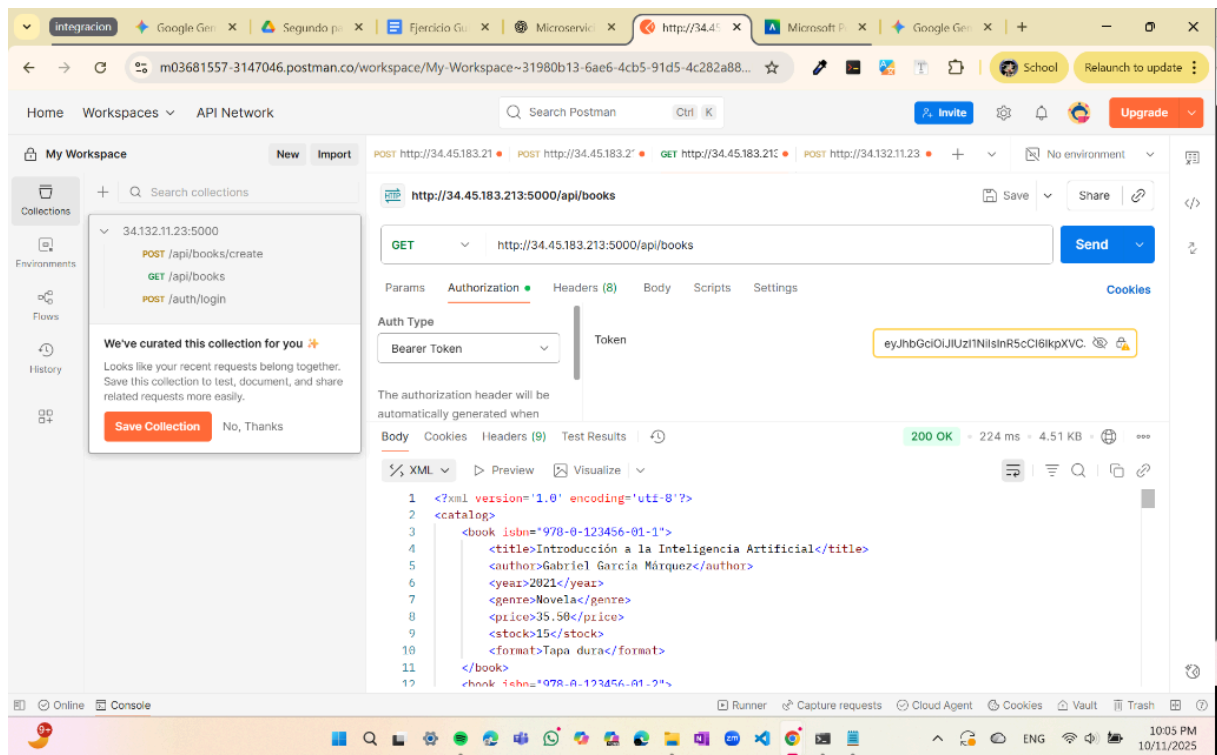
Antes de poder correr el microservicio editado, se volvió a instalar JWT para Flask en el directorio *micro02* con el comando **pip install Flask-JWT-Extended passlib** así como la librería de CORS con el comando **pip install Flask-Cors**. El comando SQL para la tabla *Users* es el siguiente:

Una vez actualizado el microservicio, se probó que este funcionará con POSTMAN al probar todas las rutas. A continuación se muestra evidencia de algunas de las pruebas:

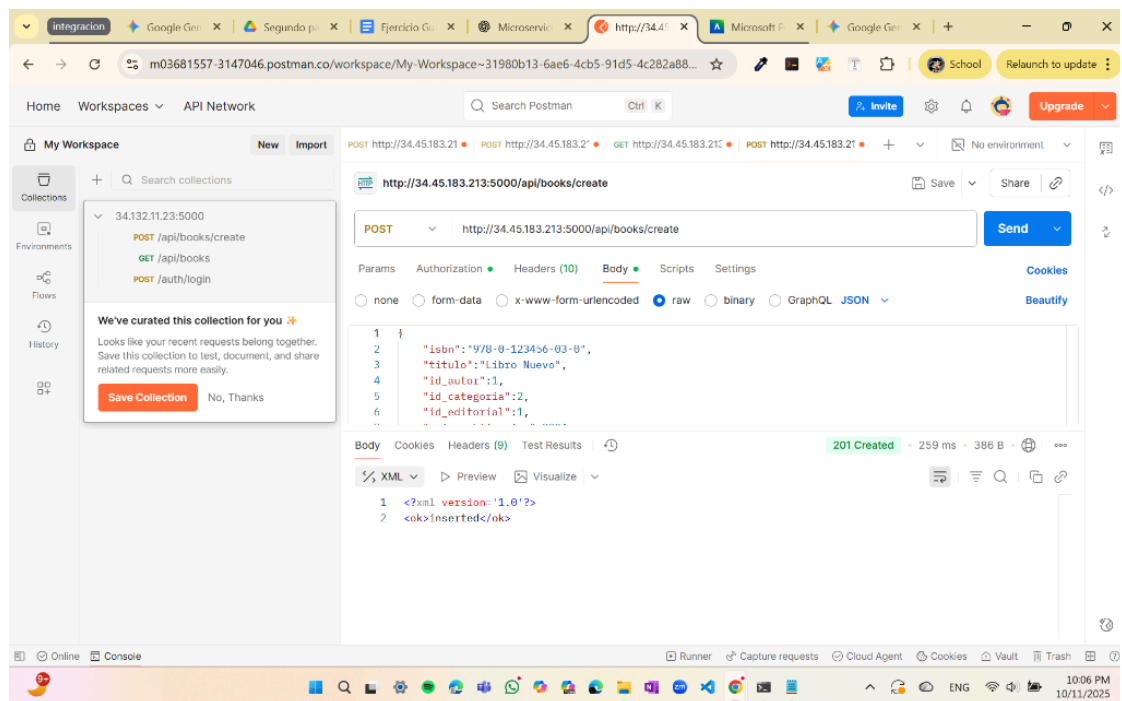
## 1. registro de usuario



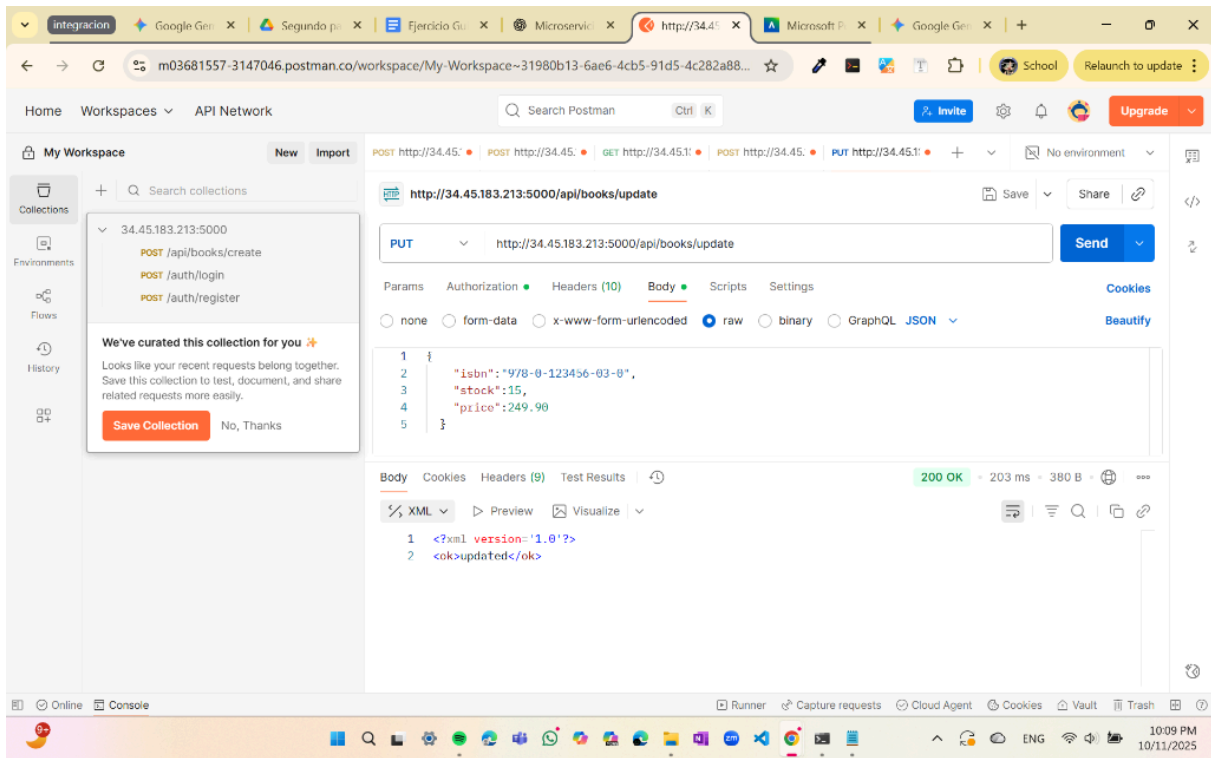




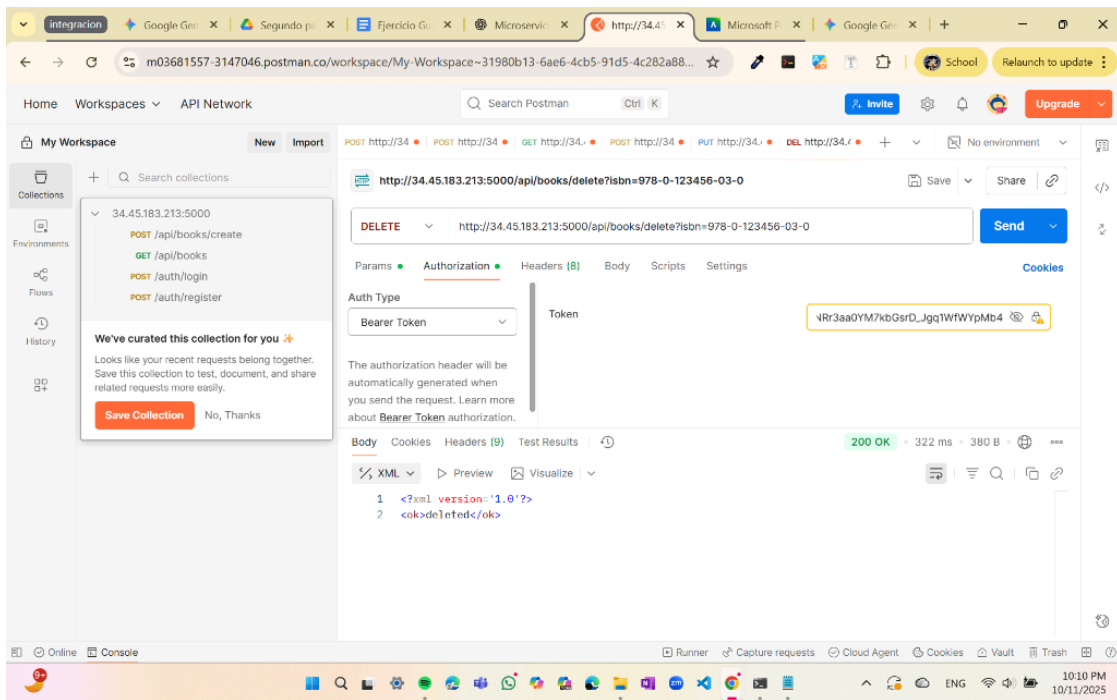
#### 4. insertar libro con el token y los datos del libro en formato JSON



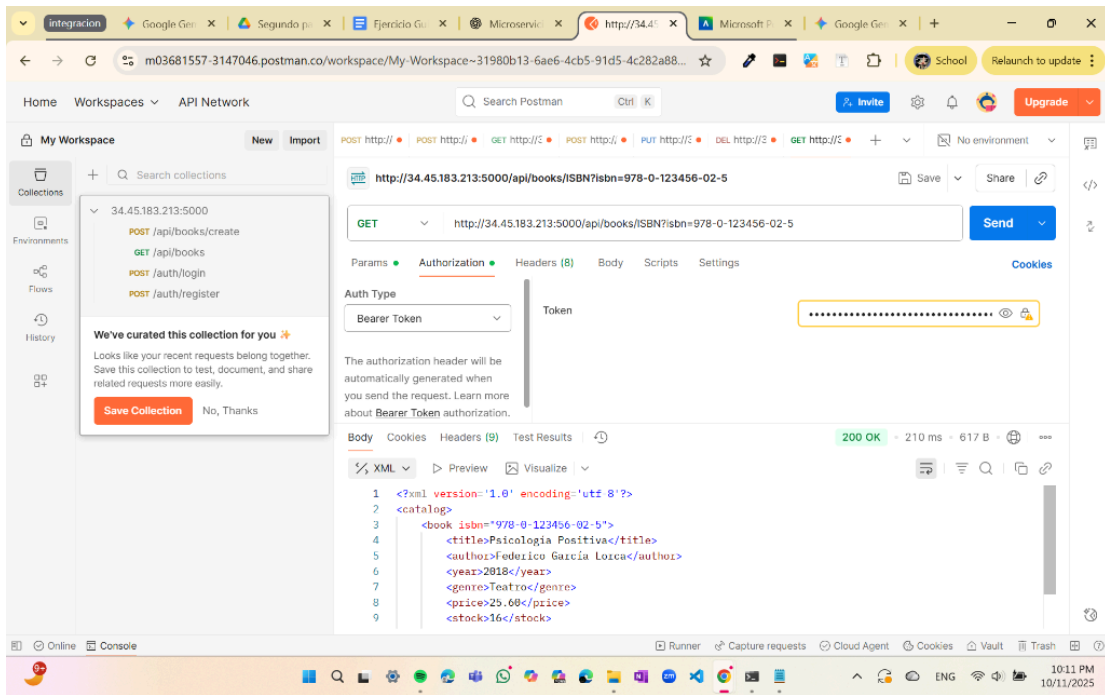
#### 5. actualizar libro con el token y los datos del libro en formato JSON



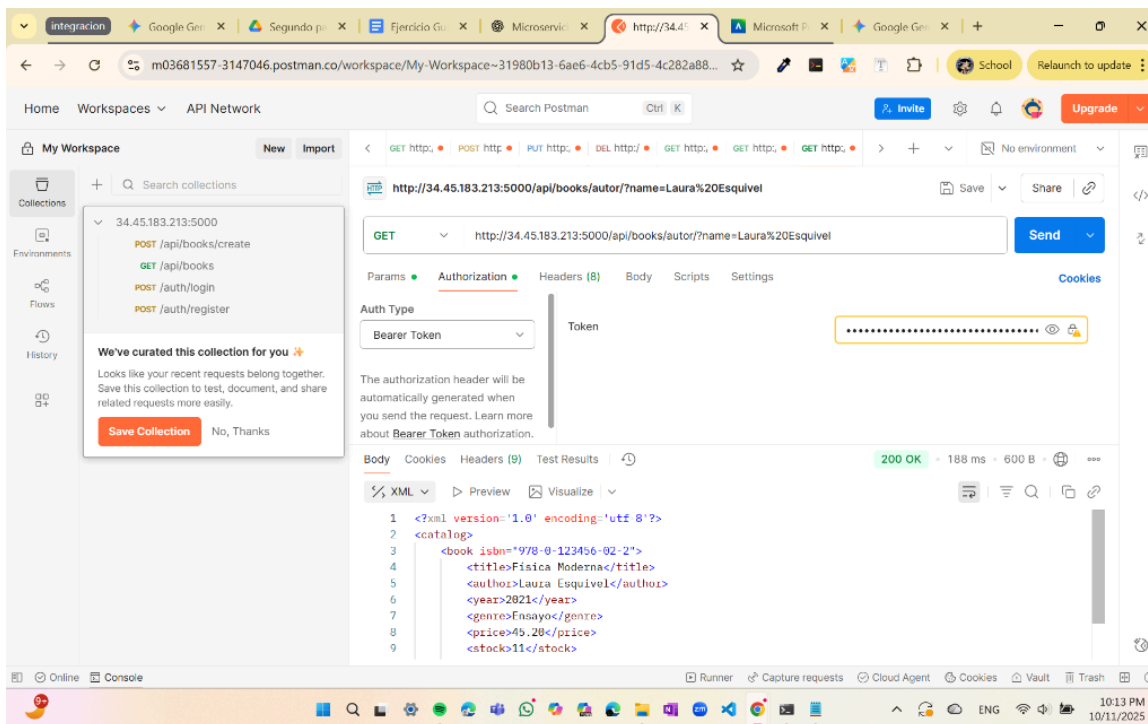
## 6. eliminar libro con el token



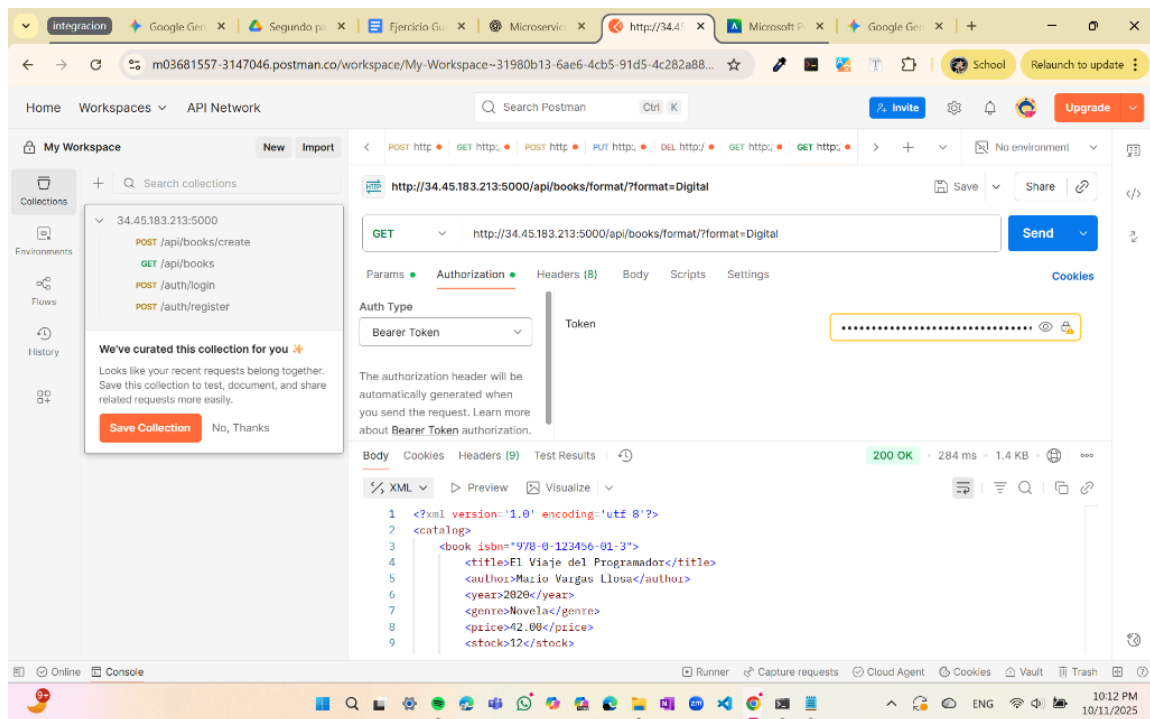
## 7. buscar por isbn



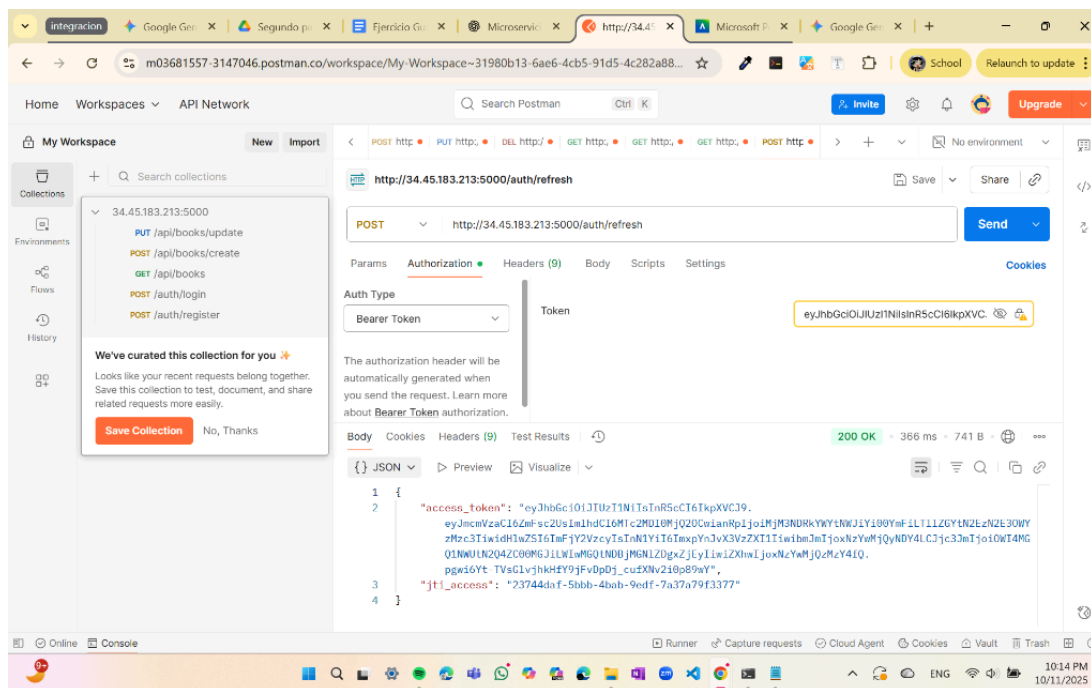
## 8. buscar por autor



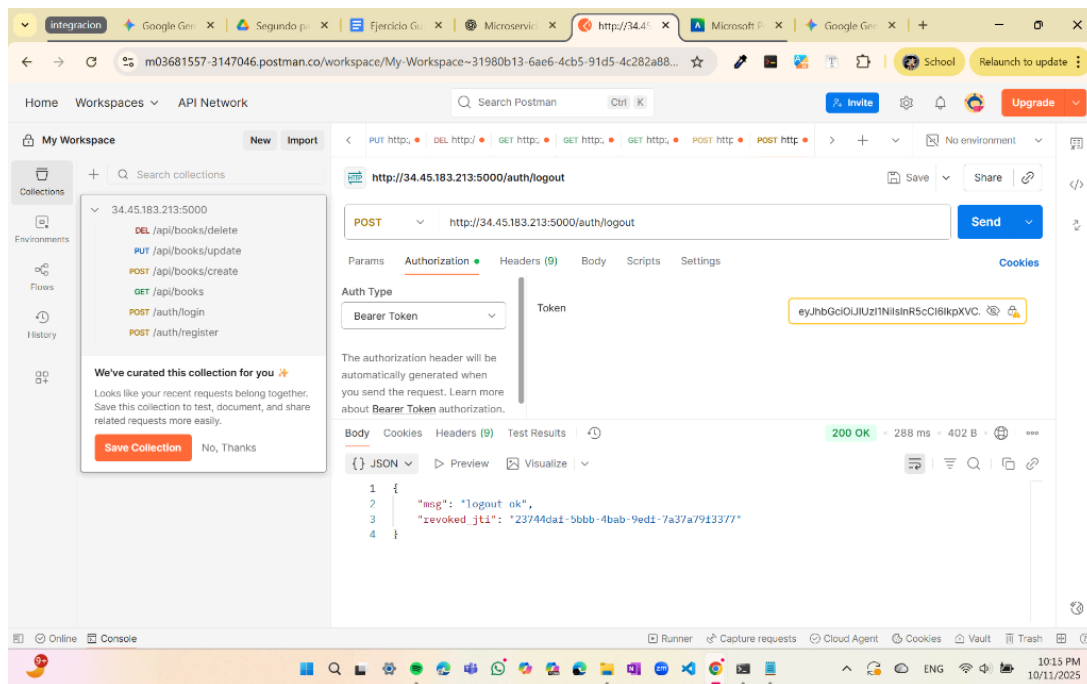
## 9. buscar por formato



## 10. hacer refresh del token



## 11. logout



Una vez que se comprobó que el microservicio funcionaba correctamente con los endpoints, se le pidió a la IA construir la interfaz gráfica de la aplicación web con el siguiente script:

*Con los archivos existentes que ya se construyeron anteriormente para el JWT, los cuales son: index.html, style.css y script.js, modifica el contenido de estos para que funcionen con el main.py actual.*

En respuesta la IA modificó solamente 1 archivo de 3, el script.js:

1. **index.html:** archivo contiene la estructura de la página web. Tendrá formularios de login, registro, búsqueda, etc. y una sección para mostrar los datos de los libros. También incluye un área de "logs" para que puedas ver lo que está sucediendo en el cliente.
2. **style.css:** archivo que proporciona un estilo básico para hacer la interfaz más atractiva y fácil de usar.
3. **script.js:** este es el archivo más importante. Contiene la lógica para enviar solicitudes, gestionar la autenticación y mostrar los resultados de los logs. Ahora funciona con los endpoints actualizados e implementa Redis además del JWT.

Código **index.html**:

```
HTML
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cliente de Microservicio de Libros</title>
```



```

    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Cliente de Microservicio de Libros</h1>

        <div class="auth-section">
            <h2>Autenticación</h2>
            <form id="login-form">
                <input type="text" id="username" placeholder="Usuario" required>
                <input type="password" id="password" placeholder="Contraseña"
required>
                <button type="submit">Iniciar Sesión</button>
            </form>
            <hr>
            <h2>Registrar Nuevo Usuario</h2>
            <form id="register-form">
                <input type="text" id="reg-username" placeholder="Nuevo Usuario"
required>
                <input type="password" id="reg-password" placeholder="Nueva
Contraseña" required>
                <button type="submit">Registrar</button>
            </form>
        </div>

        <div class="app-section hidden" id="app-section">
            <div class="section-container">
                <div class="form-section">
                    <h3>Catálogo de Libros</h3>
                    <button id="get-books">Obtener Todos los Libros</button>
                    <div id="books-container"></div>
                    <hr>

                    <h3>Buscar por ISBN</h3>
                    <form id="search-isbn-form" class="search-form">
                        <input type="text" id="isbn-search" placeholder="ISBN del
libro" required>
                        <button type="submit">Buscar por ISBN</button>
                    </form>

                    <h3>Buscar por autor</h3>
                    <form id="search-author-form" class="search-form">
                        <input type="text" id="author-search" placeholder="Nombre del
autor" required>
                        <button type="submit">Buscar por Autor</button>
                    </form>

                    <h3>Buscar por formato</h3>
                    <form id="search-format-form" class="search-form">
                        <input type="text" id="format-search" placeholder="Formato
(e.g., Paperback)" required>
                        <button type="submit">Buscar por Formato</button>

```



```

        </form>
    </div>

    <div class="form-section">
        <h3>Insertar libro</h3>
        <form id="insert-form" class="crud-form">
            <input type="text" id="isbn-insert" placeholder="ISBN"
required>
            <input type="text" id="titulo-insert" placeholder="Título"
required>
            <input type="number" id="id_autor-insert" placeholder="ID
Autor" required>
            <input type="number" id="id_categoria-insert" placeholder="ID
Categoría" required>
            <input type="number" id="id_editorial-insert" placeholder="ID
Editorial" required>
            <input type="number" id="anio_publicacion-insert"
placeholder="Año Publicación" required>
            <input type="number" id="price-insert" placeholder="Precio"
step="0.01" required>
            <input type="number" id="stock-insert" placeholder="Stock"
required>
            <input type="text" id="formato-insert" placeholder="Formato"
required>
            <button type="submit">Insertar Libro</button>
        </form>

        <h3>Actualizar libro</h3>
        <form id="update-form" class="crud-form">
            <input type="text" id="isbn-update" placeholder="ISBN a
actualizar" required>
            <input type="text" id="titulo-update" placeholder="Nuevo
Título">
            <input type="number" id="id_autor-update" placeholder="Nuevo
ID Autor">
            <input type="number" id="id_categoria-update"
placeholder="Nuevo ID Categoría">
            <input type="number" id="id_editorial-update"
placeholder="Nuevo ID Editorial">
            <input type="number" id="anio_publicacion-update"
placeholder="Nuevo Año Publicación">
            <input type="number" id="price-update" placeholder="Nuevo
Precio" step="0.01">
            <input type="number" id="stock-update" placeholder="Nuevo
Stock">
            <input type="text" id="formato-update" placeholder="Nuevo
Formato">
            <button type="submit">Actualizar Libro</button>
        </form>

        <h3>Eliminar libro</h3>
        <form id="delete-form" class="crud-form">

```

```

        <input type="text" id="isbn-delete" placeholder="ISBN a
eliminar" required>
        <button type="submit">Eliminar Libro</button>
    </form>
</div>
</div>
</div>

<div class="log-section">
    <h2>Logs del Cliente</h2>
    <pre id="client-logs"></pre>
</div>
</div>

<script src="script.js"></script>
</body>
</html>

```

## Código **style.css**:

```

CSS
body {
    font-family: 'Arial', sans-serif;
    background-color: #f0f2f5;
    color: #333;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    min-height: 100vh;
    margin: 0;
    padding: 20px;
}

.container {
    background: #fff;
    padding: 40px;
    border-radius: 12px;
    box-shadow: 0 6px 15px rgba(0, 0, 0, 0.1);
    width: 100%;
    max-width: 1200px;
    box-sizing: border-box;
}

h1, h2, h3 {
    color: #0056b3;
    text-align: center;
    margin-bottom: 20px;
}

h1 {

```

```

    font-size: 2.5em;
    border-bottom: 2px solid #0056b3;
    padding-bottom: 10px;
    margin-bottom: 30px;
}

h2 {
    font-size: 1.8em;
}

h3 {
    font-size: 1.4em;
    margin-top: 30px;
    margin-bottom: 15px;
}

.auth-section, .app-section {
    margin-bottom: 30px;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.hidden {
    display: none;
}

/* Flexbox para alinear las secciones */
.section-container {
    display: flex;
    gap: 30px; /* Espacio entre las columnas */
    flex-wrap: wrap; /* Permite que las columnas se envuelvan en pantallas pequeñas */
}

.form-section {
    flex: 1; /* Hace que cada sección ocupe el mismo espacio */
    min-width: 300px;
    padding: 20px;
    border: 1px solid #eee;
    border-radius: 8px;
    background-color: #f9f9f9;
}

.search-form, .crud-form {
    display: flex;
    flex-direction: column;
    gap: 10px;
    margin-bottom: 20px;
}

.kpis {
    display: flex;

```

```

    gap: 15px;
    margin: 20px 0;
    flex-wrap: wrap;
}

.kpi {
    background: #eef6ff;
    border: 1px solid #cce0ff;
    border-radius: 8px;
    padding: 10px 14px;
    font-size: 0.9em;
    flex: 1 1 200px;
}

.toolbar {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
    flex-wrap: wrap;
}

input[type="text"],
input[type="password"],
input[type="number"] {
    width: 100%;
    padding: 12px;
    border-radius: 6px;
    border: 1px solid #ccc;
    box-sizing: border-box;
    font-size: 1em;
}

button {
    background-color: #007bff;
    color: white;
    font-weight: bold;
    padding: 12px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.3s ease;
    font-size: 1.1em;
}

button:hover {
    background-color: #0056b3;
}

hr {
    border: 0;
    height: 1px;

```

```

        background-image: linear-gradient(to right, rgba(0, 0, 0, 0), rgba(0, 0, 0, 0.25),
        rgba(0, 0, 0, 0));
        margin: 40px 0;
    }

    #books-container {
        margin-top: 25px;
        padding: 20px;
        border: 1px solid #eee;
        border-radius: 8px;
        background-color: #fafafa;
    }

    .book-item {
        background-color: #e9ecef;
        padding: 15px;
        border-radius: 6px;
        margin-bottom: 10px;
        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
    }

    #client-logs {
        background-color: #2c2c2c;
        color: #00ff00;
        padding: 15px;
        border-radius: 8px;
        white-space: pre-wrap;
        font-family: 'Courier New', Courier, monospace;
        max-height: 250px;
        overflow-y: scroll;
        line-height: 1.4;
    }

```

### Código **script.js**:

```

JavaScript
// =====
// Config
// =====
const API_URL = 'http://34.45.183.213:5000'; // <-- cambiarla por la IP actual

// =====
// Selectores (según tu index.html)
// =====
const loginForm      = document.getElementById('login-form');
const registerForm   = document.getElementById('register-form');
const appSection     = document.getElementById('app-section');
const getBooksBtn    = document.getElementById('get-books');
const booksContainer = document.getElementById('books-container');
const clientLogs     = document.getElementById('client-logs');

```

```

// Opcionales (si existen en tu HTML)
const statusBtn = document.getElementById('status-btn');
const logoutBtn = document.getElementById('logout-btn');
const kpiAccess = document.getElementById('kpi-access');
const kpiRefresh = document.getElementById('kpi-refresh');

// Búsquedas
const searchIsbnForm = document.getElementById('search-isbn-form');
const searchFormatForm = document.getElementById('search-format-form');
const searchAuthorForm = document.getElementById('search-author-form');

// CRUD
const insertForm = document.getElementById('insert-form');
const updateForm = document.getElementById('update-form');
const deleteForm = document.getElementById('delete-form');

// =====
// Utils UI + Tokens
// =====
const log = (message, type = 'info') => {
  const timestamp = new Date().toLocaleTimeString();
  clientLogs.textContent += `[${timestamp}] [${type.toUpperCase()}] ${message}\n`;
  clientLogs.scrollTop = clientLogs.scrollHeight;
};

const redact = (tok) => (!tok ? '-' : (tok.length <= 20 ? tok :
`${tok.slice(0,10)}...${tok.slice(-10)}`));

const showApp = (show) => {
  const auth = document.querySelector('.auth-section');
  if (show) { auth?.classList.add('hidden'); appSection?.classList.remove('hidden'); }
  else { appSection?.classList.add('hidden'); auth?.classList.remove('hidden'); }
};

const updateKpis = () => {
  kpiAccess && (kpiAccess.textContent =
redact(localStorage.getItem('access_token')));
  kpiRefresh && (kpiRefresh.textContent =
redact(localStorage.getItem('refresh_token')));
};

const saveTokens = ({ access_token, refresh_token }) => {
  if (access_token) localStorage.setItem('access_token', access_token);
  if (refresh_token) localStorage.setItem('refresh_token', refresh_token);
  updateKpis();
};

const clearTokens = () => {
  localStorage.removeItem('access_token');
  localStorage.removeItem('refresh_token');
  updateKpis();
};

```

```

};

const getAccess = () => localStorage.getItem('access_token');
const getRefresh = () => localStorage.getItem('refresh_token');

// =====
// Auto-Refresh (refresh token)
// =====
const doRefresh = async () => {
  const rt = getRefresh();
  if (!rt) { log('No hay refresh token para refrescar.', 'warning'); return false; }
  try {
    const res = await fetch(`${API_URL}/auth/refresh`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${rt}` }
    });
    const data = await res.json().catch(() => null);
    if (!res.ok || !data?.access_token) {
      log(`Refresh falló: ${res.status} ${JSON.stringify(data)}`, 'error');
      return false;
    }
    saveTokens({ access_token: data.access_token });
    log('Refresh OK → nuevo access token guardado.', 'success');
    return true;
  } catch (e) {
    log(`Error de red en refresh: ${e.message}`, 'error');
    return false;
  }
};

async function fetchWithAuth(path, options = {}, retry = true) {
  const access = getAccess();
  const headers = Object.assign({}, options.headers || {}, access ? { 'Authorization': `Bearer ${access}` } : {});
  const opts = Object.assign({}, options, { headers });

  const res = await fetch(`${API_URL}${path}`, opts);

  if (res.status === 401 && retry && getRefresh()) {
    log(`401 en ${path}. Intentando refresh...`, 'warning');
    const ok = await doRefresh();
    if (ok) return fetchWithAuth(path, options, false);
    log('No fue posible refrescar. Volviendo a login...', 'error');
    clearTokens();
    showApp(false);
  }

  return res;
}

// =====
// Auth

```

```
// =====
loginForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value.trim();
  log('Intentando iniciar sesión...', 'info');

  try {
    const res = await fetch(`${API_URL}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await res.json().catch(()=>null);
    if (res.ok && data?.access_token && data?.refresh_token) {
      saveTokens({ access_token: data.access_token, refresh_token: data.refresh_token
    });
    log('¡Inicio de sesión exitoso!', 'success');
    showApp(true);
  } else {
    log(`Error de autenticación: ${data?.msg || res.status}`, 'error');
  }
} catch (err) {
  log(`Error de red en login: ${err.message}`, 'error');
}
});

registerForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('reg-username').value.trim();
  const password = document.getElementById('reg-password').value.trim();
  log('Intentando registrar usuario...', 'info');

  try {
    const res = await fetch(`${API_URL}/auth/register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await res.json().catch(()=>null);
    if (res.ok) {
      log('¡Registro exitoso! Ya puedes iniciar sesión.', 'success');
    } else {
      log(`Error de registro: ${data?.msg || res.status}`, 'error');
    }
  } catch (err) {
    log(`Error de red en registro: ${err.message}`, 'error');
  }
});

logoutBtn?.addEventListener('click', async () => {
  log('Cerrando sesión...', 'info');
```



```

    const res = await fetchWithAuth('/auth/logout', { method: 'POST' }, /*retry*/
false);
    try {
        const data = await res.json().catch(()=>null);
        if (res.ok) log(`Logout OK.`, 'success');
        else log(`Logout devolvió ${res.status}: ${JSON.stringify(data)}`, 'warning');
    } catch { /* ignore */}
    clearTokens();
    showApp(false);
});

// =====
// Books (XML render)
// =====
function displayBooks(xmlDoc) {
    booksContainer.innerHTML = '';
    const books = xmlDoc.getElementsByTagName('book');
    if (!books || books.length === 0) {
        booksContainer.innerHTML = '<p>No se encontraron libros.</p>';
        return;
    }
    for (const book of books) {
        const isbn = book.getAttribute('isbn');
        const title = book.getElementsByTagName('title')[0]?.textContent;
        const authors =
Array.from(book.getElementsByTagName('author')).map(a=>a.textContent).join(', ');
        const year = book.getElementsByTagName('year')[0]?.textContent;
        const genre = book.getElementsByTagName('genre')[0]?.textContent;
        const price = book.getElementsByTagName('price')[0]?.textContent;
        const stock = book.getElementsByTagName('stock')[0]?.textContent;
        const format = book.getElementsByTagName('format')[0]?.textContent;

        const div = document.createElement('div');
        div.classList.add('book-item');
        div.innerHTML = `
            <strong>ISBN:</strong> ${isbn || 'Desconocido'}<br>
            <strong>Título:</strong> ${title || 'Desconocido'}<br>
            <strong>Autores:</strong> ${authors || 'Desconocido'}<br>
            <strong>Año:</strong> ${year || 'Desconocido'}<br>
            <strong>Género:</strong> ${genre || 'Desconocido'}<br>
            <strong>Precio:</strong> ${price || 'Desconocido'}<br>
            <strong>Stock:</strong> ${stock || 'Desconocido'}<br>
            <strong>Formato:</strong> ${format || 'Desconocido'}
        `;
        booksContainer.appendChild(div);
    }
}

// =====
// Acciones: listado y búsquedas
// =====
getBooksBtn?.addEventListener('click', async () => {

```

```

log('Solicitando lista de libros...', 'info');
try {
  const res = await fetchWithAuth('/api/books', { method: 'GET' });
  const xmlText = await res.text();
  if (res.ok) {
    const xmlDoc = new DOMParser().parseFromString(xmlText, 'text/xml');
    displayBooks(xmlDoc);
    log('¡Solicitud de libros exitosa!', 'success');
  } else {
    log(`Error al obtener libros: ${res.status} - ${xmlText.slice(0,200)}`,
    'error');
  }
} catch (err) {
  log(`Error de red al obtener libros: ${err.message}`, 'error');
}
});

searchIsbnForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const isbn = document.getElementById('isbn-search').value.trim();
  if (!isbn) return;
  log(`Buscando por ISBN: ${isbn}...`, 'info');
  try {
    const res = await
fetchWithAuth(`/api/books/ISBN?isbn=${encodeURIComponent(isbn)}`, { method: 'GET' });
    const xmlText = await res.text();
    if (res.ok) {
      const xmlDoc = new DOMParser().parseFromString(xmlText, 'text/xml');
      displayBooks(xmlDoc);
      log('Búsqueda por ISBN exitosa.', 'success');
    } else {
      log(`Error en búsqueda ISBN: ${res.status} - ${xmlText.slice(0,200)}`, 'error');
    }
  } catch (err) {
    log(`Error de red en búsqueda ISBN: ${err.message}`, 'error');
  }
});

searchFormatForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const format = document.getElementById('format-search').value.trim();
  if (!format) return;
  log(`Buscando por formato: ${format}...`, 'info');
  try {
    const res = await
fetchWithAuth(`/api/books/format/?format=${encodeURIComponent(format)}`, { method:
'GET' });
    const xmlText = await res.text();
    if (res.ok) {
      const xmlDoc = new DOMParser().parseFromString(xmlText, 'text/xml');
      displayBooks(xmlDoc);
      log('Búsqueda por formato exitosa.', 'success');
    }
  }
});

```

```

    } else {
      log(`Error en búsqueda formato: ${res.status} - ${xmlText.slice(0,200)}`,
        'error');
    }
  } catch (err) {
    log(`Error de red en búsqueda formato: ${err.message}`, 'error');
  }
});

searchAuthorForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const author = document.getElementById('author-search').value.trim();
  if (!author) return;
  log(`Buscando por autor: ${author}...`, 'info');
  try {
    const res = await
fetchWithAuth(`/api/books/autor/?name=${encodeURIComponent(author)}`, { method: 'GET'
});
    const xmlText = await res.text();
    if (res.ok) {
      const xmlDoc = new DOMParser().parseFromString(xmlText, 'text/xml');
      displayBooks(xmlDoc);
      log('Búsqueda por autor exitosa.', 'success');
    } else {
      log(`Error en búsqueda autor: ${res.status} - ${xmlText.slice(0,200)}`,
        'error');
    }
  } catch (err) {
    log(`Error de red en búsqueda autor: ${err.message}`, 'error');
  }
});

// =====
// CRUD
// =====
insertForm?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const payload = {
    isbn: document.getElementById('isbn-insert').value.trim(),
    titulo: document.getElementById('titulo-insert').value.trim(),
    id_autor: parseInt(document.getElementById('id_autor-insert').value, 10),
    id_categoria: parseInt(document.getElementById('id_categoria-insert').value, 10),
    id_editorial: parseInt(document.getElementById('id_editorial-insert').value, 10),
    anio_publicacion:
parseInt(document.getElementById('anio_publicacion-insert').value, 10),
    price: parseFloat(document.getElementById('price-insert').value),
    stock: parseInt(document.getElementById('stock-insert').value, 10),
    formato: document.getElementById('formato-insert').value.trim(),
  };
  log('Insertando libro...', 'info');
  try {
    const res = await fetchWithAuth('/api/books/create', {

```

```

        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
    });
    const text = await res.text();
    if (res.ok) log('Libro insertado con éxito.', 'success');
    else log(`Error insert: ${res.status} - ${text.slice(0,200)}`, 'error');
} catch (err) {
    log(`Error de red en insert: ${err.message}`, 'error');
}
});

updateForm?.addEventListener('submit', async (e) => {
    e.preventDefault();
    const payload = {
        isbn: document.getElementById('isbn-update').value.trim(),
        titulo: document.getElementById('titulo-update').value.trim(),
        id_autor: document.getElementById('id_autor-update').value.trim(),
        id_categoria: document.getElementById('id_categoria-update').value.trim(),
        id_editorial: document.getElementById('id_editorial-update').value.trim(),
        anio_publicacion: document.getElementById('anio_publicacion-update').value.trim(),
        price: document.getElementById('price-update').value.trim(),
        stock: document.getElementById('stock-update').value.trim(),
        formato: document.getElementById('formato-update').value.trim(),
    };
    Object.keys(payload).forEach(k => payload[k] === '' && delete payload[k]);
    log('Actualizando libro...', 'info');
    try {
        const res = await fetchWithAuth('/api/books/update', {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const text = await res.text();
        if (res.ok) log('Libro actualizado con éxito.', 'success');
        else log(`Error update: ${res.status} - ${text.slice(0,200)}`, 'error');
    } catch (err) {
        log(`Error de red en update: ${err.message}`, 'error');
    }
});

deleteForm?.addEventListener('submit', async (e) => {
    e.preventDefault();
    const isbn = document.getElementById('isbn-delete').value.trim();
    if (!isbn) return;
    log('Eliminando libro...', 'info');
    try {
        const res = await
fetchWithAuth(`/api/books/delete?isbn=${encodeURIComponent(isbn)}`, {
            method: 'DELETE'
        });
        const text = await res.text();

```

```

        if (res.ok) log('Libro eliminado con éxito.', 'success');
        else log(`Error delete: ${res.status} - ${text.slice(0,200)}`, 'error');
    } catch (err) {
        log(`Error de red en delete: ${err.message}`, 'error');
    }
});

// =====
// Estado (opcional): botón "Estado de Autenticación"
// =====
statusBtn?.addEventListener('click', async () => {
    const token = getAccess();
    if (!token) { log('Sin access token en localStorage.', 'warning'); return; }
    // Hacemos una llamada ligera para verificar validez del access:
    const res = await fetchWithAuth('/api/books', { method: 'GET' });
    if (res.ok) log('Estado: autenticado (access válido).', 'success');
    else log(`Estado: token inválido/expirado (${res.status}).`, 'warning');
});

// =====
// Init
// =====
(function init() {
    const hasTokens = !!getAccess() && !!getRefresh();
    showApp(hasTokens);
    updateKpis();
    if (hasTokens) log('Sesión restaurada desde localStorage.', 'info');
})();

```

Estos tres códigos se copiaron y pegaron en sus respectivos archivos, con el mismo nombre, en el nuevo directorio *WebApplication2* el cual está separado y aislado del directorio donde se encuentran los microservicios.

Luego se tuvo que crear otra regla de firewall para que el navegador pudiera mostrar la página web sin que saliera error de conexión o acceso, dicha regla es la siguiente:

```

C:\Users\mccco>gcloud compute firewall-rules create allow-http-web-app-2 --network=default --direction=INGRESS --priority=1000 --action=ALLOW --rules=tcp:8080 --source-ranges=0.0.0.0/0
Creating firewall...Created [https://www.googleapis.com/compute/v1/projects/integracion-468300/global/firewalls/allow-http-web-app-2].
Creating firewall...done.

```

| NAME                 | NETWORK | DIRECTION | PRIORITY | ALLOW    | DENY | DISABLED |
|----------------------|---------|-----------|----------|----------|------|----------|
| allow-http-web-app-2 | default | INGRESS   | 1000     | tcp:8080 |      | False    |

Para ejecutar la página web, se tiene que ejecutar el microservicio dentro de su directorio *Microservices/micro02* con los comandos **export FLASK\_APP=main.py** y **flask run --host=0.0.0.0**. Dentro del directorio de *WebApplication2*, se escribe el comando **sudo python3 -m http.server 8080**.

```

Installing collected packages: Flask-Cors
Successfully installed Flask-Cors-6.0.1
[m03681557@maquina01 micro01]$ export FLASK_APP=main.py
[m03681557@maquina01 micro01]$ flask run --host=0.0.0.0
 * Serving Flask app 'main.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.128.0.3:5000
Press CTRL+C to quit

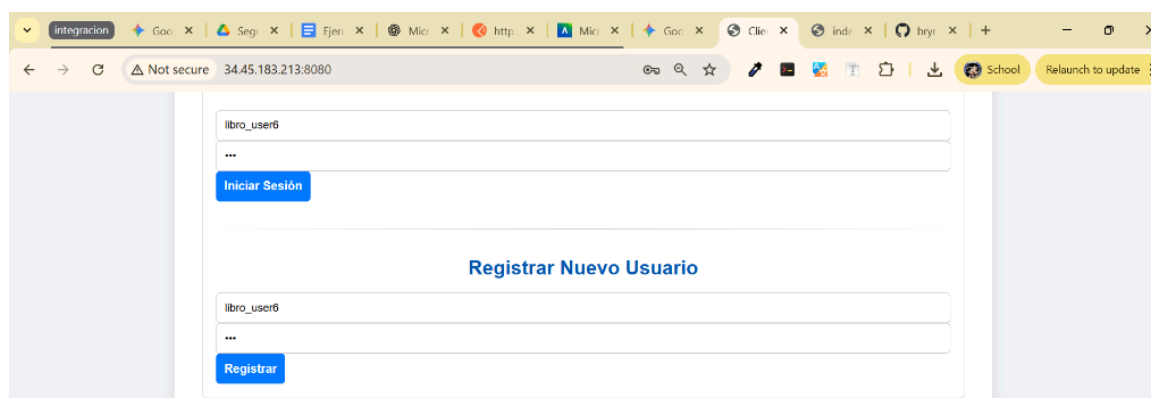
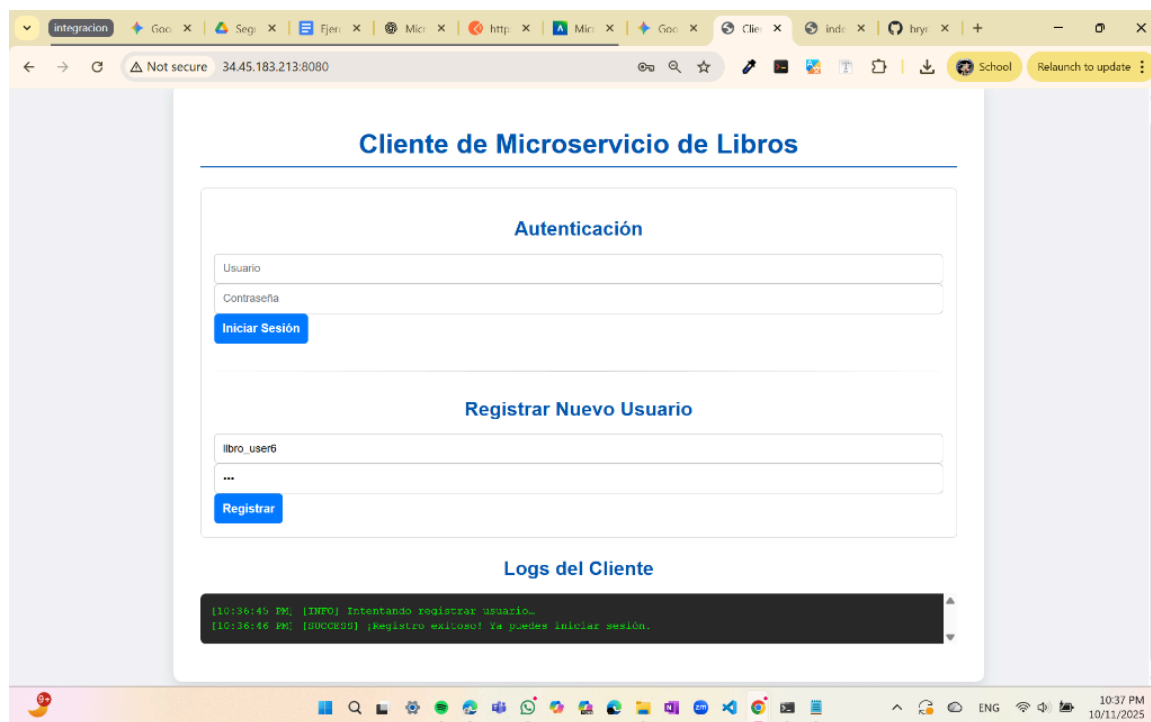
```

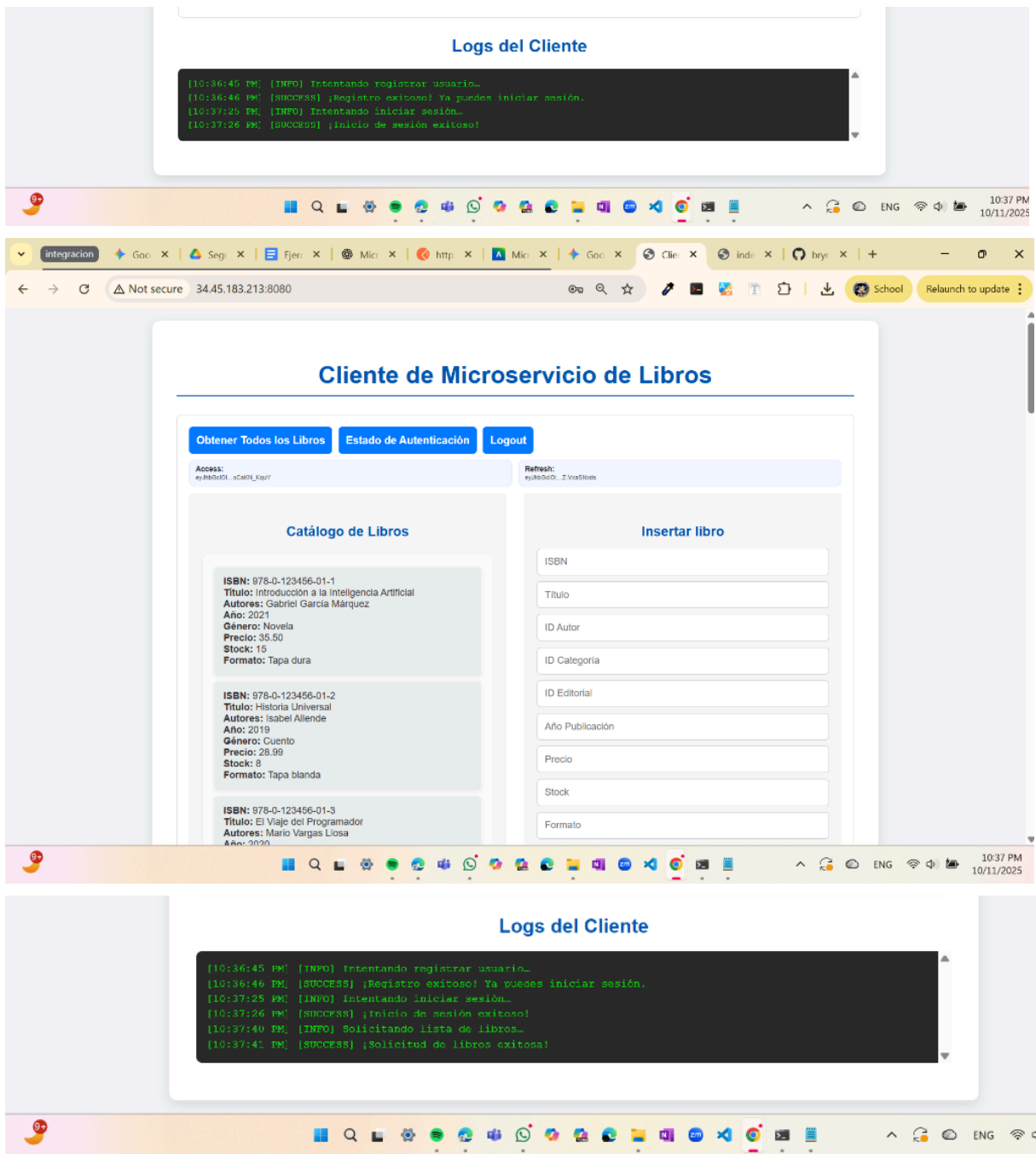
```

[m03681557@maquina01 WebApplication]$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET / HTTP/1.1" 304 -
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET /style.css HTTP/1.1" 304 -
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET /script.js HTTP/1.1" 200 -

```

Por último, se muestran fotos del funcionamiento de la página web resultante:









integracion

Goo XSegi XFjerrMicXhttpXMicXGoo XClie Xindé Xhry X

Not secure34.45.183.213:8080

SchoolRelaunch to update

Obtener Todos los LibrosEstado de AutenticaciónLogout

Access:  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...

Refresh:  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...

Catálogo de Libros

ISBN: 978-0-123456-01-7  
Titulo: Diseño UX/UI  
Autores: Jorge Luis Borges  
Año: 2023  
Género: Novela  
Precio: 48.75  
Stock: 18  
Formato: Digital

Buscar por ISBN

ISBN del libro

Buscar por ISBN

Buscar por autor

Jorge Luis Borges

Buscar por Autor

Insertar libro

ISBN

Titulo

ID Autor

ID Categoría

ID Editorial

Año Publicación

Precio

Stock

Formato

Insertar Libro

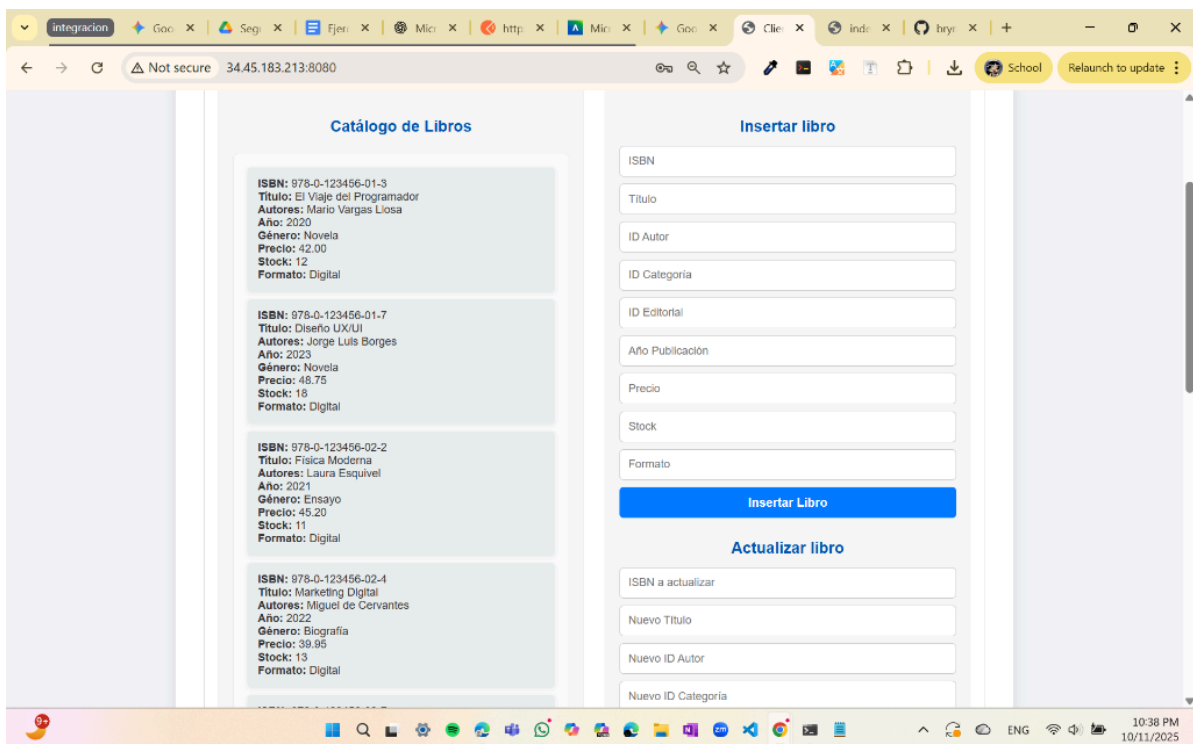
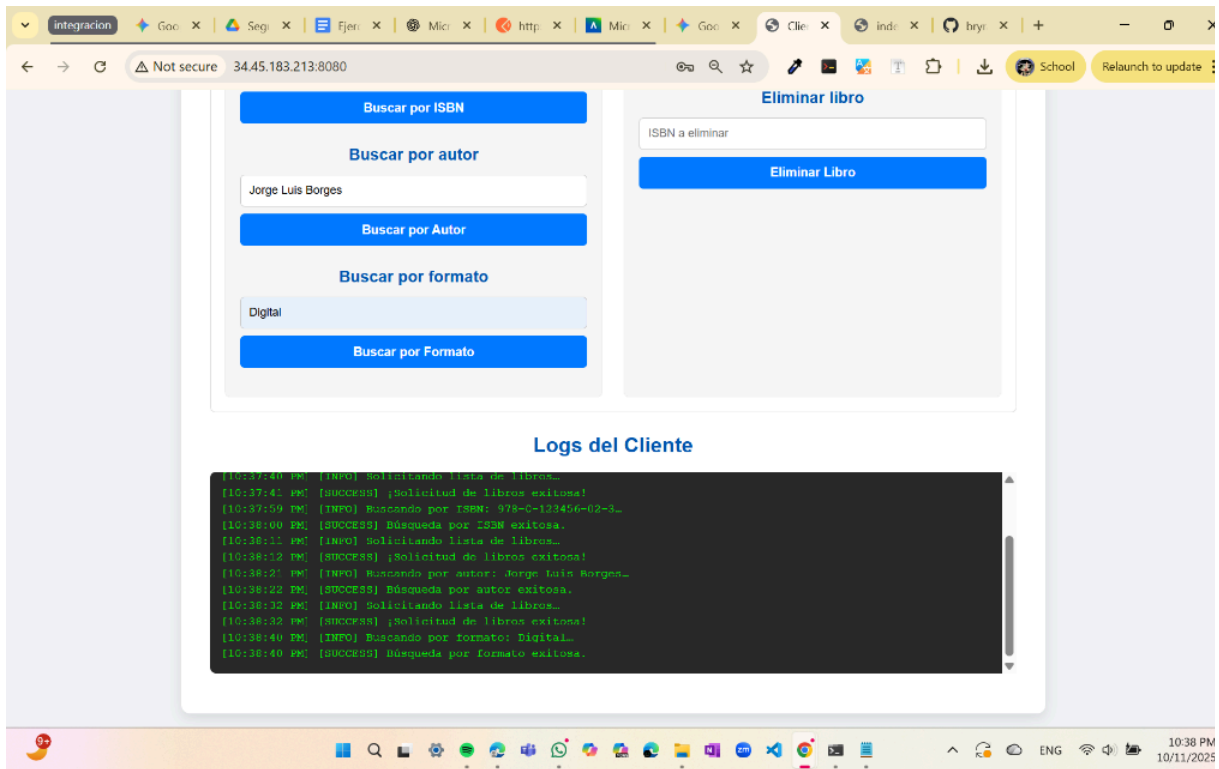
Actualizar libro

ISBN a actualizar

Logs del Cliente

```
[10:36:45 PM] [INFO] Intentando registrar usuario...
[10:36:46 PM] [SUCCESS] ¡Registro exitoso! Ya puedes iniciar sesión.
[10:37:25 PM] [INFO] Intentando iniciar sesión.
[10:37:26 PM] [SUCCESS] ¡Inicio de sesión exitoso!
[10:37:40 PM] [INFO] Solicitando lista de libros...
[10:37:41 PM] [SUCCESS] ¡Solicitud de libros exitosa!
[10:37:42 PM] [INFO] Buscando por ISBN: 978-0-123456-02-3...
[10:37:59 PM] [SUCCESS] Búsqueda por ISBN exitosa.
[10:38:00 PM] [INFO] Solicitando lista de libros...
[10:38:12 PM] [SUCCESS] ¡Solicitud de libros exitosa!
[10:38:12 PM] [INFO] Buscando por autor: Jorge Luis Borges...
[10:38:22 PM] [SUCCESS] Búsqueda por autor exitosa.
```

36



integracion x Goo x Segi x Ejerc x Mic x http x A Mic x Goo x Clien x ind x bry x +

Not secure 34.45.183.213:8080

## Cliente de Microservicio de Libros

[Obtener Todos los Libros](#) [Estado de Autenticación](#) [Logout](#)

**Access:** myBibGestCli - ac6a24 KeyV

**Refresh:** myBibGestCli - 2-10ad50e0a

### Catálogo de Libros

**ISBN:** 978-0-123456-01-3  
**Título:** El Viaje del Programador  
**Autores:** Mario Vargas Llosa  
**Año:** 2020  
**Género:** Novela  
**Precio:** 42.00  
**Stock:** 12  
**Formato:** Digital

**ISBN:** 978-0-123456-01-7  
**Título:** Diseño UX/UI  
**Autores:** Jorge Luis Borges  
**Año:** 2023  
**Género:** Novela  
**Precio:** 48.75  
**Stock:** 18  
**Formato:** Digital

**ISBN:** 978-0-123456-02-2  
**Título:** Física Moderna  
**Autores:** Laura Esquivel  
**Año:** 2021  
**Género:** Ensayo  
**Precio:** 45.20  
**Stock:** 11

### Insertar libro

978-0321765723

The Lord of the Rings: The Fellowship of the Rin

1

1

1

32

0.17

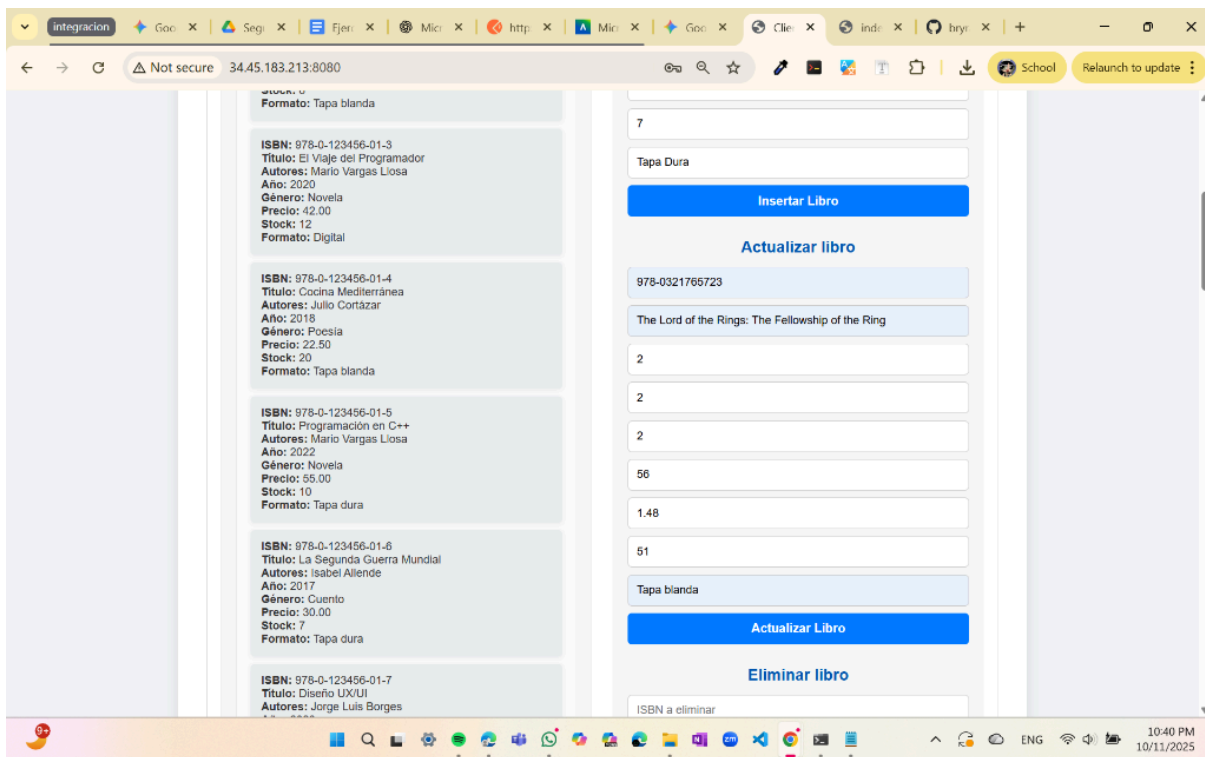
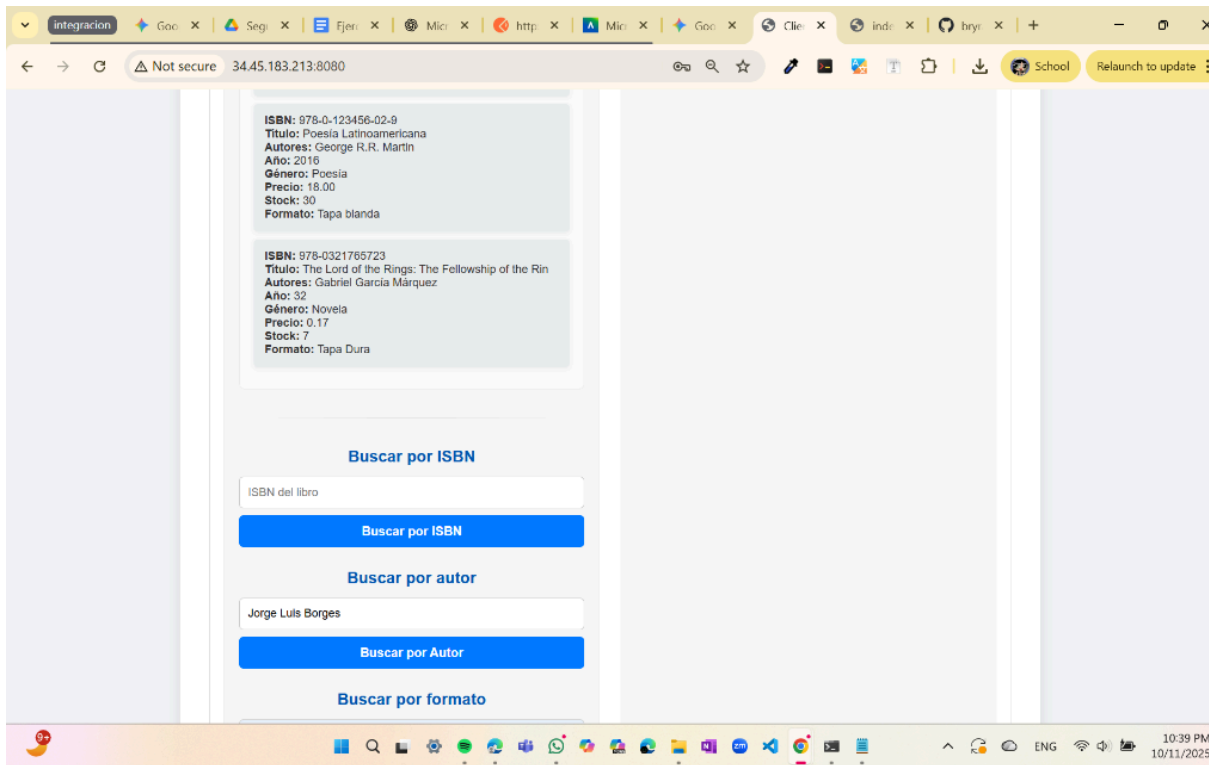
7

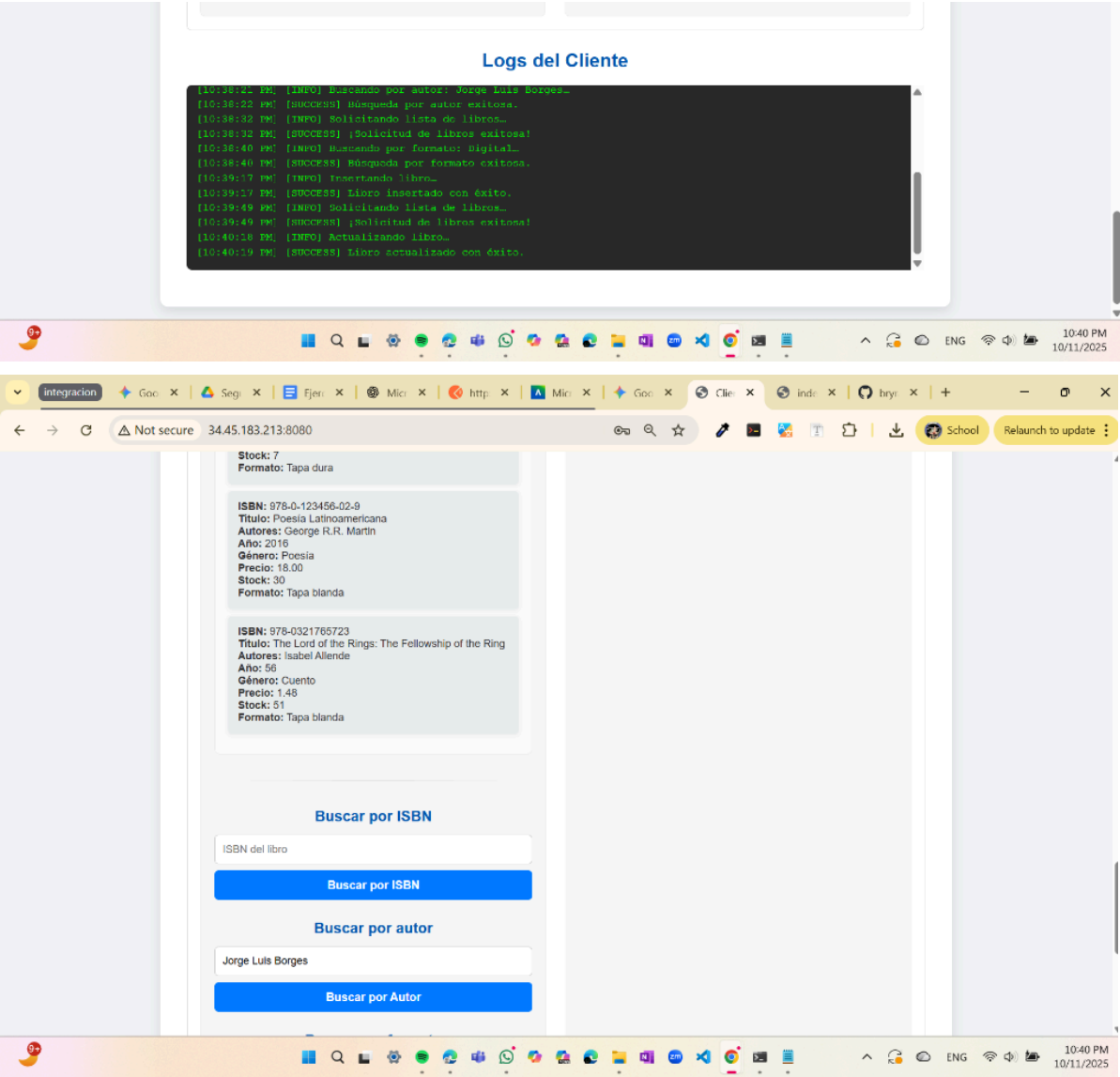
Tapa Dura

[Insertar Libro](#)

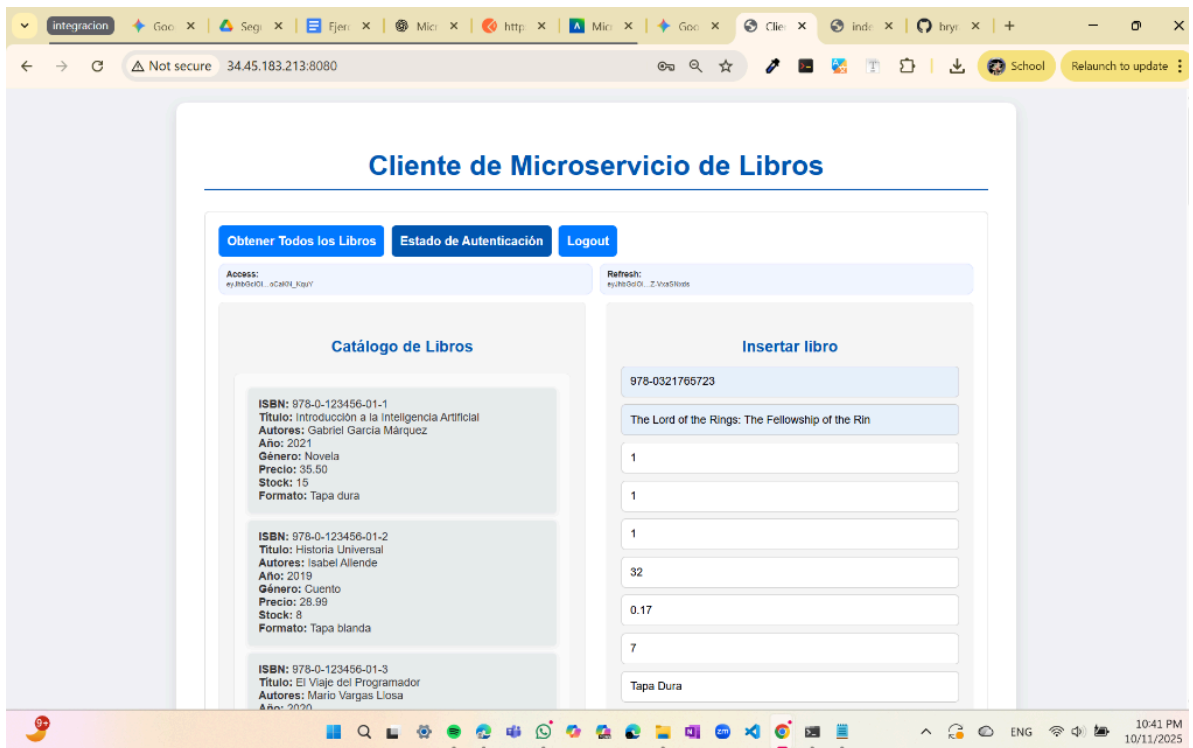
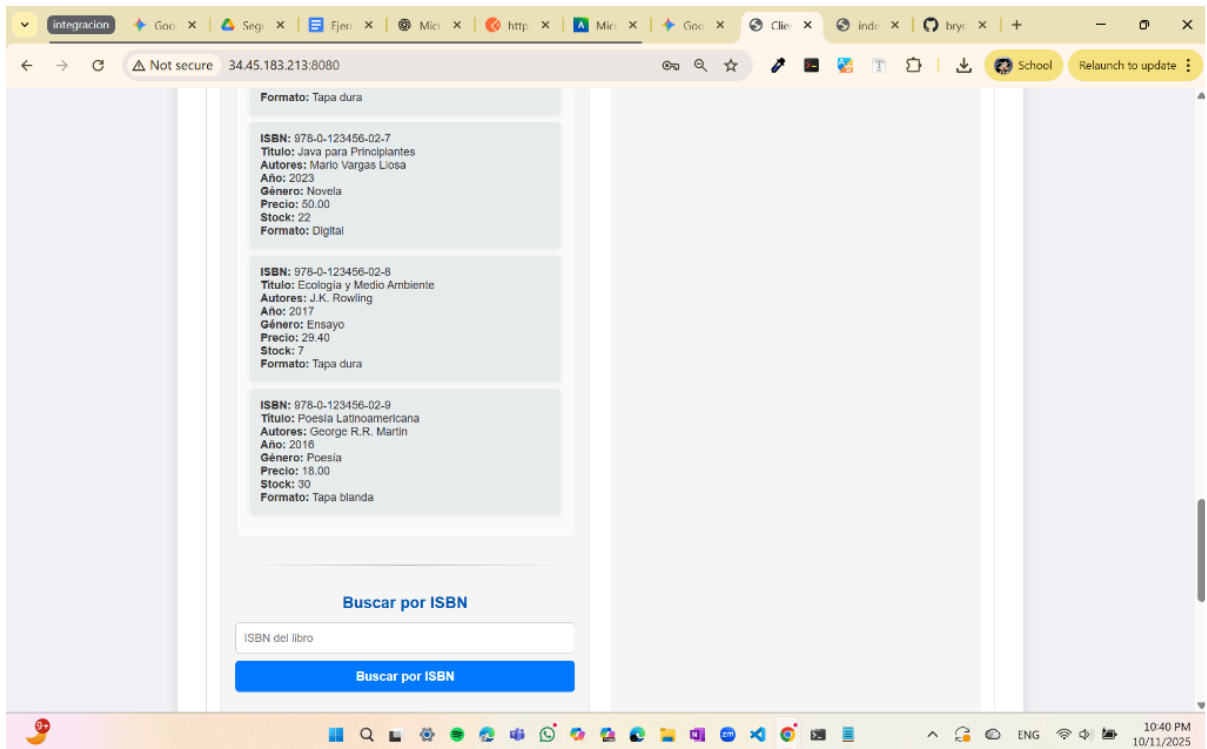
## Logs del Cliente

```
[10:37:59 PM] [INFO] Buscando por ISBN: 978-0-123456-02-2...
[10:38:00 PM] [SUCCESS] Búsqueda por ISBN exitosa.
[10:38:11 PM] [INFO] Solicitando lista de libros...
[10:38:12 PM] [SUCCESS] Solicitud de libros exitosa!
[10:38:21 PM] [INFO] Buscando por autor: Jorge Luis Borges...
[10:38:22 PM] [SUCCESS] Búsqueda por autor exitosa.
[10:38:32 PM] [INFO] Solicitando lista de libros...
[10:38:32 PM] [SUCCESS] Solicitud de libros exitosa!
[10:38:40 PM] [INFO] Buscando por formato: Digital...
[10:38:40 PM] [SUCCESS] Búsqueda por formato exitosa.
[10:38:47 PM] [INFO] Insertando libro...
[10:38:47 PM] [SUCCESS] Libro insertado con éxito.
```









### Logs del Cliente

```

[10:39:17 PM] [SUCCESS] Libro insertado con éxito.
[10:39:49 PM] [INFO] Solicitando lista de libros...
[10:39:49 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:18 PM] [INFO] Actualizando libro...
[10:40:19 PM] [SUCCESS] Libro actualizado con éxito.
[10:40:20 PM] [INFO] Solicitando lista de libros...
[10:40:20 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:37 PM] [INFO] Eliminando libro...
[10:40:37 PM] [SUCCESS] Libro eliminado con éxito.
[10:40:45 PM] [INFO] Solicitando lista de libros...
[10:40:46 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:53 PM] [SUCCESS] Estado: autenticado (accesos válidos).

```

### Logs del Cliente

```

[10:39:49 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:18 PM] [INFO] Actualizando libro...
[10:40:19 PM] [SUCCESS] Libro actualizado con éxito.
[10:40:20 PM] [INFO] Solicitando lista de libros...
[10:40:20 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:37 PM] [INFO] Eliminando libro...
[10:40:37 PM] [SUCCESS] Libro eliminado con éxito.
[10:40:45 PM] [INFO] Solicitando lista de libros...
[10:40:46 PM] [SUCCESS] Solicitudes de libros exitosa!
[10:40:53 PM] [SUCCESS] Estado: autenticado (accesos válidos).
[10:41:10 PM] [INFO] Cerrando sesión...
[10:41:10 PM] [SUCCESS] Logout: OK.

```

## Consideraciones importantes

1. La estructura del directorio del ejercicio es el siguiente:

/ (directorio raíz)

├── Microservices/

│ └── micro02/

.



```
|      └─ main.py
|
└─ WebApplication/
    |
    └─ index.html
    |
    └─ style.css
    |
    └─ script.js
```

2. Los **tokens los refresca en automático el microservicio cada 30 días**. En caso de inactividad el usuario se sale del sitio web y solo se puede ingresar si el usuario ya está registrado e ingresa su nombre de usuario y contraseñas correctas. De lo contrario, no se puede acceder.
3. Se tuvo que **manipular el código del script.js** para que al mostrar los libros se vieran todos los atributos así como el **index.html** y **style.css** para que se mostraran de manera correcta y ordenada y no hubiera problemas con los endpoints o CORS.

## Conclusión

Esta práctica me permitió reforzar y ampliar mi entendimiento sobre microservicios seguros y escalables. La integración de Redis al sistema de autenticación con JWT no solo mejoró el control de sesiones, sino que también me dio una visión más profunda sobre la revocación de tokens, refresh automático y validación estricta mediante listas en memoria.

Uno de los principales retos fue lograr que el endpoint de registro (`/auth/register`) funcionara correctamente desde el cliente web. El problema radicaba en la forma en que el archivo `script.js` enviaba la solicitud al backend; fue necesario ajustar manualmente la URL y asegurar que los headers y el payload fueran compatibles con la API. Este obstáculo me enseñó la importancia de alinear la comunicación entre el cliente y el servidor, especialmente cuando se trabaja con microservicios desplegados en distintos entornos.

En resumen, este ejercicio fue una experiencia completa que abarcó desde la implementación técnica hasta la solución de problemas reales en el despliegue. El uso conjunto de Flask, JWT, Redis, MariaDB y una interfaz web personalizada me proporcionó una base sólida para enfrentar proyectos más complejos en el ámbito de los microservicios modernos y seguros.